

AgroSense: Technical Documentation

This document provides a comprehensive technical overview of the AgroSense hardware and software stack.

1. System Architecture

The system consists of two primary components:

- AgroSense Node (Hardware):** An ESP8266 (NodeMCU) equipped with environmental sensors.
- AgroSense Dashboard (Software):** A desktop application built with JavaFX and Maven.

These components communicate via a Local Area Network (LAN). The desktop application hosts an embedded, lightweight HTTP Server (`SensorIngestServer`) on port `8080` that actively listens for incoming sensor telemetry from paired nodes.

2. Hardware Wiring (NodeMCU ESP8266)

- Temperature & Humidity Sensor (DHT11)**
 - `VCC` -> `VIN` (5V power is required for stable DHT11 readings)
 - `GND` -> `GND`
 - `DATA` -> `D4` (GPIO2)
- Capacitive Soil Moisture Sensor v1.2**
 - `VCC` -> `3V3`
 - `GND` -> `GND`
 - `AOUT` -> `A0` (Analog Input)
- Hardware Reset Trigger**
 - Pin `D1` (GPIO5). Shorting this pin to `GND` during boot forces the NodeMCU to clear its EEPROM memory and restart in Provisioning (Hotspot) mode.

3. Communication Protocol & Data Ingestion

The NodeMCU connects to the local WiFi and sends an `HTTP POST` request to the Java application's Ingest Server every 30 seconds.

Endpoint: `http://<java-server-ip>:8080/ingest` **Content-Type:** `application/x-www-form-urlencoded` **Payload structure:**

```
pairingCode=0055-112F&temp=31.5&humidity=84.8&soil=35.0
```

The Java application validates the `pairingCode` against the database. If it corresponds to a registered `DeviceUnit` , it parses the values and inserts them into the `SensorReading` table.

4. Software Stack (JavaFX)

- UI Framework:** JavaFX 21
- Database:** SQLite (`agrosense.db`)
- Build Tool:** Maven

Concurrency and Real-time Updates

The JavaFX application is designed to be highly responsive and real-time:

- **Background Ingestion:** `SensorIngestServer` runs on a separate, dedicated background thread pool so incoming HTTP requests do not block the UI.
- **Auto-Refresh:** The `DashboardController` utilizes a JavaFX `Timeline` to poll the database every 10 seconds. This avoids manual reloading and safely updates the UI thread with the latest `SensorReadings`.
- **Status Detection:** The Dashboard dynamically determines if a node is "Online" or "Offline" by calculating the time delta between the current time and the most recent `SensorReading`. If `delta <= 2 minutes`, the node is flagged as Online.

5. Device Provisioning & Pairing Flow

1. **Access Point Mode:** When unconfigured, the ESP8266 hosts a WiFi hotspot named `AgroSense-XXXX-XXXX` (where XXXX-XXXX is the unique pairing code).
2. **Provisioning:** The user connects to this hotspot and navigates to `192.168.4.1`, entering their local home WiFi credentials and the Java Server's IP address.
3. **Connection:** The ESP8266 restarts, connects to the home WiFi, and immediately begins sending data to the Java Server.
4. **App Pairing:** In the Java App, the user clicks "Pair Device" and types in the code `XXXX-XXXX`. The app creates a `Site`, links the `DeviceUnit` to the user's account, and allows the data to appear on the Dashboard.

6. Database Schema Summary (SQLite)

- `Customer` : Stores user login accounts.
- `Site` : Represents a physical location (e.g., "Tomato Garden"). Can exist independently of a hardware device.
- `DeviceUnit` : Represents the physical ESP8266 node.
- `DevicePairing` : An associative table linking a `Customer`, `Site`, and `DeviceUnit`. Supports history tracking (`is_active` flag, `paired_date`, `unpaired_date`).
- `SensorReading` : Raw telemetry data (`TEMPERATURE`, `HUMIDITY`, `SOIL_MOISTURE`), foreign-keyed to the `DeviceUnit`.
- `AlertRule` / `AlertEvent` : System for generating threshold alerts (e.g., "Humidity > 80%").

AgroSense Design Patterns

This document justifies the major design patterns used in the AgroSense platform.

1. Chain of Responsibility Pattern

Location: `com.agrosense.pairing.*`

Problem: The process of pairing a device using a code requires multiple sequential validations:

1. Does the code exist?
2. Is the code expired?
3. Has the code already been used?
4. Is the physical device available (not faulty)?
5. Is the device already paired?

Solution: The Chain of Responsibility pattern is used to decouple these checks. Each validation step is encapsulated in its own handler (e.g., `CodeExistsHandler`, `CodeNotExpiredHandler`). **Benefit:** We can

easily add, remove, or reorder validation steps without modifying a monolithic `if-else` block in the `PairingService` .

2. State Pattern

Location: `com.agrosense.state.*`

Problem: Entities like `DeviceUnit` have lifecycles that dictate their behavior (e.g. `UNPAIRED` -> `PAIRED` -> `FAULTY`). Hardcoding conditionals based on status strings creates brittle code that's hard to maintain.

Solution: The State pattern represents each lifecycle phase as a distinct class (`UnpairedState` , `PairedState` , `FaultyState`). **Benefit:** State transitions are controlled explicitly by the state classes. Illegal transitions (like moving an already `PAIRED` device directly back to `UNPAIRED` without unpairing) throw standard exceptions defined by the interface.

3. Strategy Pattern

Location: `com.agrosense.strategy.*`

Problem: An alert behaves differently depending on the `UseCaseProfile` of the `Site` (`HOME` vs `FARM`). A home garden might just log an event, while a farm plot might trigger immediate escalation.

Solution: The Strategy pattern encapsulates the alert response logic into profile-specific classes (`HomeAlertStrategy` , `FarmAlertStrategy`). **Benefit:** When a new customer tier is added (e.g. `COMMERCIAL_GREENHOUSE`), we simply implement a new `AlertResponseStrategy` without touching the core `AlertService` evaluation loop.

4. Observer Pattern

Location: `com.agrosense.observer.*`

Problem: When a new `AlertEvent` is generated, multiple disconnected components need to react. For example, the live JavaFX Dashboard needs to increment a counter, and a background logger needs to record the event.

Solution: The Observer pattern (`AlertObserver`) allows the `AlertService` to broadcast events to a list of subscribers (`DashboardAlertObserver` , `LogAlertObserver`). **Benefit:** The `AlertService` doesn't need to know anything about the UI or the logging system. It simply notifies registered observers, keeping the domains strictly decoupled.